

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: Implement best security practices for IP, email, and web service using PGP,S/MIME for enhancing email Security.CO (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS:** 25

Annexure A

Coding Challenge

Code of Conduct:

I E. Lakshmi Narayanan(192524249) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Lakshmi Narayanan

1. Secure Email Transmission using S/MIME

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
    public_exponent=65537,
    key_size=2048
)
receiver_public_key = receiver_private_key.public_key()

# Original email
message = b"Hello, this is a confidential email."

# Generate AES session key
aes_key = os.urandom(32)
iv = os.urandom(16)

# Encrypt email using AES
cipher = Cipher(algorithms.AES(aes_key), modes.CFB(iv))
encryptor = cipher.encryptor()
encrypted_message = encryptor.update(message) + encryptor.finalize()

# Encrypt AES key using receiver's RSA public key
encrypted_aes_key = receiver_public_key.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Create digital signature using sender's private key
signature = sender_private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    )
```

```

        hashes.SHA256()
    )

    print("Original Message:")
    print(message.decode())

    print("\nEncrypted Message:")
    print(encrypted_message.hex())

    print("\nDigital Signature Generated Successfully.")

    # Receiver decrypts AES key
    decrypted_aes_key = receiver_private_key.decrypt(
        encrypted_aes_key,
        padding.OAEP(
            mgf=padding.MGF1(hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

    # Decrypt email
    cipher = Cipher(algorithms.AES(decrypted_aes_key), modes.CFB(iv))
    decryptor = cipher.decryptor()
    decrypted_message = decryptor.update(encrypted_message) + decryptor.finalize()

    print("\nDecrypted Message:")
    print(decrypted_message.decode())

    # Verify signature
    try:
        sender_public_key.verify(
            signature,
            decrypted_message,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
    
```

```
print("\nDigital Signature: VERIFIED")
except Exception:
    print("\nDigital Signature: INVALID")
```

```
C:\Users\USER\OneDrive\Documents>python smime_email.py
C:\Users\USER\OneDrive\Documents\smime_email.py:27: CryptographyDeprecationWarning: CFB has been moved to cryptography.hazmat.decrepit.cip
hers.modes.CFB and will be removed from cryptography.hazmat.primitives.ciphers.modes in 49.0.0.
    cipher = Cipher(algorithms.AES(aes_key), modes.CFB(iv))
Original Message:
Hello, this is a confidential email.

Encrypted Message:
6ad96fb6f983f6ee06e54e695e3651c97deb1958e9bbf398f238c70a580bb7cblac54adf

Digital Signature Generated Successfully.
C:\Users\USER\OneDrive\Documents\smime_email.py:70: CryptographyDeprecationWarning: CFB has been moved to cryptography.hazmat.decrepit.cip
hers.modes.CFB and will be removed from cryptography.hazmat.primitives.ciphers.modes in 49.0.0.
    cipher = Cipher(algorithms.AES(decrypted_aes_key), modes.CFB(iv))
Decrypted Message:
Hello, this is a confidential email.

Digital Signature: VERIFIED
C:\Users\USER\OneDrive\Documents>
```

Analysis

- **Confidentiality:** AES encrypts the email contents.
- **Key protection:** RSA encrypts the AES session key.
- **Authentication:** Sender signs the message using their private key.
- **Integrity:** Signature verification detects message modification.
- **S/MIME:** In a real system, certificates bind public keys to identities.

2. PGP-Based File Encryption and Decryption

This program demonstrates the main PGP concepts: **key generation, key distribution, encryption, decryption, and authentication.**

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
    modes.CFB(iv)
)

encryptor = cipher.encryptor()
encrypted_file = encryptor.update(file_data) + encryptor.finalize()

# Encrypt session key using RSA public key
encrypted_session_key = public_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# -----
# DIGITAL SIGNATURE
# -----

signature = private_key.sign(
    file_data,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nFile Encrypted Successfully.")
```

```

print("Session Key Encrypted.")
print("Digital Signature Generated.")

# -----
# DECRYPTION
# -----

decrypted_session_key = private_key.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

cipher = Cipher(
    algorithms.AES(decrypted_session_key),
    modes.CFB(iv)
)

decryptor = cipher.decryptor()
decrypted_file = decryptor.update(encrypted_file) + decryptor.finalize()

print("\nDecrypted File:")
print(decrypted_file.decode())

# -----
# SIGNATURE VERIFICATION
# -----

try:
    public_key.verify(
        signature,
        decrypted_file,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

```

)

```
print("\nPGP Authentication: SUCCESS")
```

```
print("File Integrity: VERIFIED")
```

except Exception:

```
print("\nAuthentication Failed!")
```

```
C:\Users\USER\OneDrive\Documents>pgp_2.py
===== PGP FILE ENCRYPTION AND DECRYPTION =====

1. PGP Key Pair Generated Successfully.

Original File Content:
This is a confidential PGP protected file.

2. Symmetric Session Key Generated.
C:\Users\USER\OneDrive\Documents\pgp_2.py:45: CryptographyDeprecationWarning: CFB has been moved to cryptography.hazmat.decrepit.ciphers.modes.CFB and will be removed from cryptography.hazmat.primitives.ciphers.modes in 49.0.0.
  modes.CFB(iv)
3. File Encrypted Successfully.
4. Session Key Encrypted Using Public Key.
5. Digital Signature Generated.

6. Session Key Decrypted Successfully.
C:\Users\USER\OneDrive\Documents\pgp_2.py:185: CryptographyDeprecationWarning: CFB has been moved to cryptography.hazmat.decrepit.ciphers.modes.CFB and will be removed from cryptography.hazmat.primitives.ciphers.modes in 49.0.0.
  modes.CFB(iv)
7. File Decrypted Successfully.

Decrypted File Content:
This is a confidential PGP protected file.

8. Digital Signature: VERIFIED
File Authentication: SUCCESS
File Integrity: VERIFIED

===== PGP PROCESS COMPLETED =====
```

Analysis

PGP normally uses **hybrid encryption**:

1. A symmetric session key encrypts the file.
2. The session key is encrypted using the recipient's public key.
3. A digital signature provides authentication and integrity.
4. Public keys can be distributed through key servers or trusted key exchange mechanisms.
5. Symmetric encryption makes PGP efficient for large files.

3. IPSec Tunnel Mode Simulation

Python Program

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

# Original IP packet
original_packet = (
    "SOURCE IP: 192.168.1.10\n"
    "DESTINATION IP: 10.0.0.20\n"
    "DATA: Confidential Network Information"
)

print("Original IP Packet:")
print(original_packet)

# Convert to bytes
packet_bytes = original_packet.encode()

# Generate symmetric encryption key
key = os.urandom(32)
iv = os.urandom(16)

# -----
# IPSec TUNNEL MODE
# -----

cipher = Cipher(
    algorithms.AES(key),
    modes.CFB(iv)
)

encryptor = cipher.encryptor()
encrypted_packet = encryptor.update(packet_bytes) + encryptor.finalize()

print("\nEncrypted IP Packet:")
print(encrypted_packet.hex())
```



```

# New outer IP header
outer_header = (
    "OUTER SOURCE: 172.16.1.1\n"
    "OUTER DESTINATION: 172.16.2.1\n"
)

print("\nIPSec Tunnel Packet:")
print(outer_header)
print("Encrypted Inner IP Packet:", encrypted_packet.hex())

# -----
# DECRYPTION AT DESTINATION
# -----

cipher = Cipher(
    algorithms.AES(key),
    modes.CFB(iv)
)

decryptor = cipher.decryptor()
decrypted_packet = decryptor.update(encrypted_packet) + decryptor.finalize()

print("\nDecrypted Inner IP Packet:")
print(decrypted_packet.decode())

```

```

C:\Users\USER\OneDrive\Documents>python pgp_3.py
===== IPSec TUNNEL MODE SIMULATION =====

Original IP Packet:
Source IP: 192.168.1.10
Destination IP: 10.0.0.20
Data: Confidential Information

Encrypted IP Packet:
gAAAAABgwwVRD2-5VEJ_M00STXIZbMYw-U_6NHum2KUIKzm7H6XqV8-cdbtyQ1IXO_TUldbtQCvdG1phxixvoh6QdK_YzvOfpC0wixAY4mq5lt689MZqZjqVx-qJS1leALenxH37Kshujc-5KnK6b8KvrcUZL2LDtEAqNfS1F2q-Uhm5v_xlnh_VirqbpScv0x_NapDrKuy

Decrypted IP Packet:
Source IP: 192.168.1.10
Destination IP: 10.0.0.20
Data: Confidential Information

IPSec Tunnel Mode: SUCCESS

```

Tunnel Mode vs Transport Mode

Feature	Tunnel Mode	Transport Mode
IP packet	Entire original packet encrypted	Only payload is encrypted
Original IP header	Encrypted	Remains visible
New IP header	Added	Normally not added
Common use	VPNs	Host-to-host communication
Security	Higher protection for original addresses	Lower header protection

Analysis

In **IPSec Tunnel Mode**, the complete original IP packet is encapsulated and encrypted. A new outer IP header is added so that routers can deliver the packet to the destination.

4. SSL/TLS Handshake Simulation

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding

print("\n3. Certificate Exchange")
print("Server sends its digital certificate/public key.")

# -----
# SESSION KEY
# -----

session_key = os.urandom(32)

# Encrypt session key using server public key
encrypted_session_key = server_public_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
```

```

        algorithm=hashes.SHA256(),
        label=None
    )
)

print("\n4. Session Key Generated")
print("Client securely sends encrypted session key.")

# Server decrypts session key
server_session_key = server_private_key.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("Server successfully obtained session key.")

# -----
# SECURE DATA TRANSFER
# -----

message = b"HTTPS Secure Data"

iv = os.urandom(16)

cipher = Cipher(
    algorithms.AES(server_session_key),
    modes.CFB(iv)
)

encryptor = cipher.encryptor()
encrypted_data = encryptor.update(message) + encryptor.finalize()

print("\n5. Encrypted Application Data:")
print(encrypted_data.hex())

# Server decrypts

```

```

cipher = Cipher(
    algorithms.AES(server_session_key),
    modes.CFB(iv)
)

decryptor = cipher.decryptor()
decrypted_data = decryptor.update(encrypted_data) + decryptor.finalize()

print("\n6. Decrypted Application Data:")
print(decrypted_data.decode())

print("\nTLS Handshake Completed Successfully.")
print("Secure communication established.")

```

```

C:\Users\USER\OneDrive\Documents>python pgp_4.py
===== SSL/TLS HANDSHAKE SIMULATION =====

1. Client Hello
Client sends supported TLS versions.

2. Server Hello
Server selects TLS version and cipher.

3. Certificate Exchange
Server sends its certificate.

4. Session Key Generated
Session Key Encrypted and Sent to Server.
Server Successfully Obtained Session Key.

5. Encrypted Data:
gAAAAABqgwYRjU-bedxHfpe2U9aJNqdjFxxPxn-F8iDADFZJ4-e7kGduAlrWRPG1IG7ByTxRmCQ-sGJmYIVg4IMQvEmorqOOSB0Yo8mX40qsN-AF_OvZmEJIirBhReNyYiqNQkYLDLmk

6. Decrypted Data:
Hello, Secure Web Communication!

===== TLS HANDSHAKE COMPLETED SUCCESSFULLY =====

```

Analysis

The TLS handshake provides:

- **Authentication** through the server certificate.
- **Confidentiality** through an encrypted session key and symmetric encryption.
- **Integrity** through cryptographic authentication mechanisms.
- **Secure communication** after the handshake.

This is a simplified educational simulation. Modern TLS, such as TLS 1.3, uses ephemeral key exchange such as ECDHE rather than simply encrypting a session key with RSA.

5. Email Phishing Detection using Python

This program extracts simple features from an email and uses a basic scoring/classification method.

Python Program

```
import re

features["keyword_count"] = keyword_count

# Suspicious URL patterns
suspicious_patterns = [
    "bit.ly",
    "tinyurl",
    "@",
    "login-",
    "verify-",
    ".xyz",
    ".tk"
]

suspicious_urls = 0

for pattern in suspicious_patterns:
    if pattern in text:
        suspicious_urls += 1

features["suspicious_url"] = suspicious_urls

# Urgency detection
urgency_words = [
    "urgent",
    "immediately",
    "within 24 hours",
    "act now"
]
```

```

urgency_count = 0

for word in urgency_words:
    if word in text:
        urgency_count += 1

features["urgency"] = urgency_count

return features

# -----
# CLASSIFICATION
# -----

def classify_email(features):

    score = 0

    score += features["url_count"] * 1
    score += features["keyword_count"] * 2
    score += features["suspicious_url"] * 3
    score += features["urgency"] * 2

    print("\nPhishing Score:", score)

    if score >= 7:
        return "PHISHING EMAIL"
    else:
        return "LEGITIMATE EMAIL"

# -----
# MAIN PROGRAM
# -----

email = input("Enter email content:\n")

features = extract_features(email)

```

```
print("\nExtracted Features:")
print("Number of URLs:", features["url_count"])
print("Suspicious Keywords:", features["keyword_count"])
print("Suspicious URL Patterns:", features["suspicious_url"])
print("Urgency Indicators:", features["urgency"])
```

```
result = classify_email(features)
```

```
print("\nClassification Result:")
print(result)
```

Sample Input

URGENT! Your bank account has been suspended.

Verify your password immediately.

Click here: <http://bit.ly/login-account>

Sample Output

Extracted Features:

Number of URLs: 1

Suspicious Keywords: 4

Suspicious URL Patterns: 1

Urgency Indicators: 2

Phishing Score: 13

Classification Result:

PHISHING EMAIL

```
C:\Users\USER\OneDrive\Documents>python pgp_5
python: can't open file 'C:\\Users\\USER\\OneDrive\\Documents\\pgp_5': [Errno 2] No such file or directory

C:\Users\USER\OneDrive\Documents>python pgp_5.py
===== EMAIL PHISHING DETECTION =====

Enter email content:
URGENT! Your bank account has been suspended. Verify your password immediately. Click here: http://bit.ly/login-account

===== EXTRACTED FEATURES =====
Number of URLs: 1
Suspicious Keywords: 6
Suspicious URL Patterns: 2
Urgency Indicators: 2

Phishing Score: 23

Classification: PHISHING EMAIL

===== PHISHING DETECTION COMPLETED =====
```

Analysis

The program checks:

- **URL patterns**
- **Suspicious keywords**
- **Urgency-related language**
- **Suspicious domains/shortened URLs**

A real phishing detector would normally use a trained machine-learning model such as **Naive Bayes, Logistic Regression, Random Forest, or a neural network**, trained on a labeled dataset. Accuracy should be evaluated using **precision, recall, F1-score, and confusion matrix**, because both false positives and false negatives matter.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding ; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases